



The A-to-Z Project Compendium & Viva Defense Guide

Everything you need to know about the Metadata-Driven Intelligent News Retrieval System: what we built, why we chose specific tools, and why we rejected alternatives.

1. The Executive Summary

What is this project? It is a locally hosted, offline-first search engine specifically built for news articles.

The Problem: Standard search bars (like a basic SQL `LIKE` query or TF-IDF) only look for exact words. They don't understand meaning, and they don't care if an article was written yesterday or 10 years ago.

The Solution: We built a "Hybrid Retrieval System". It reads the exact text, uses AI to understand the meaning, and checks metadata (like publish date and author credibility) to rank the absolute best articles for the user in under 100 milliseconds.

2. The Data Ingestion Layer (Where the data comes from)

What we do: We use a static baseline dataset (AG News) and combine it with live APIs (Guardian, Mediastack).

- **Why AG News?** It's a gold-standard academic dataset with ~120,000 articles perfectly categorized into 4 classes (World, Sports, Business, Sci/Tech). It proves our system scales locally.
 - **Why The Guardian & Mediastack APIs?** AG News is "frozen" in time. We needed to prove our system handles *streaming* real-time data. The Guardian provides high-credibility prestige journalism, and Mediastack provides a firehose of global aggregator news.
 - **Why background polling?** If we fetched data while the user searched, the UI would freeze. We run fetching asynchronously in the background so the user never has to wait.
-

3. The Preprocessing Pipeline (Cleaning the text)

Before we can search, we must clean the text.

- **Lowercasing & Regex Cleaning:** Removes punctuation, HTML tags, and converts everything to lowercase. *Why?* Because to a computer, "Apple" and "apple!" are two completely different words. This reduces our total vocabulary size.
 - **Stopword Removal:** Removes words like "the", "is", "at". *Why?* They take up database space and provide zero semantic meaning.
 - **Stemming (Porter Stemmer):** Converts words to their root (e.g., "running" → "run").
 - *Why did we choose Stemming over Lemmatization?* Lemmatization is more accurate (it knows "better" comes from "good"), but it is extremely slow. Stemming just chops the ends off words, which is lightning fast and perfect for processing 120,000 articles locally.
-

4. The Lexical Engine (The "Exact Match" Layer)

What we use: BM25 (Best Matching 25)

- **What is it?** It is an advanced, probabilistic evolution of TF-IDF. It counts how many times a search term appears in an article.
 - **Why we chose BM25:** It handles **Document Length Normalization**. If the word "Economic" appears 5 times in a 10-word tweet, that's highly relevant. If it appears 5 times in a 1,000-page book, it's not relevant. BM25 understands this ratio perfectly. Next to this, it is unmatched at finding *exact proper nouns* (like "Elon Musk").
 - **Why didn't we ONLY use BM25?** The "Vocabulary Mismatch Problem." If an article is about an "Automobile", and the user searches for "Car", BM25 gives a score of 0 because it only strictly matches letters.
-

5. The Semantic Engine (The "Meaning" Layer)

What we use: `all-MiniLM-L6-v2` (Transformer) + FAISS (Vector Database)

- **What is MiniLM?** It is an AI model that reads a paragraph and outputs a 384-dimensional mathematical vector (a list of 384 numbers). Articles with similar meanings fall close to each other in mathematical space.
- **Why MiniLM instead of OpenAI or heavy BERT?** OpenAI costs money and requires the cloud. Giant 768-dimensional BERT models require expensive GPUs. MiniLM is lightweight, highly optimized, and runs perfectly on standard laptop CPUs—fitting our “offline-first” architecture perfectly.
- **Why FAISS (Facebook AI Similarity Search)?**
 - *The alternative we rejected:* Standard Cosine Similarity. If we had 120,000 articles, doing the math to compare a query against *every single article* in a standard loop takes seconds/minutes. It’s $O(N)$ complexity.
 - *Why FAISS won:* FAISS uses Approximate Nearest Neighbors (ANN). It clusters vectors into “neighborhoods,” so it only has to search a tiny fraction of the database. It compares vectors in sub-milliseconds. $O(\log N)$ complexity.
- **Why Incremental Indexing?** Originally, every time a new article arrived, we rebuilt the FAISS index from scratch. This took 1 hour. We switched to incremental indexing (appending only new vectors to the existing index) which takes seconds.

6. The Metadata Engine (The “Real-World Context” Layer)

- **Recency:** News rots fast. An article about the “Presidential Election” from 2012 is useless in 2024. We use a **Time-Decay Function** that slowly lowers the score of older articles.
- **Source Authority:** Fake news is a massive problem. We assign credibility multipliers (e.g., The Guardian = 1.2x boost, unknown blog = 0.8x drop).

7. The Core Algorithm (The Hybrid Fusion Formula)

The moment of truth: combining BM25, FAISS, and Metadata into one final rank.

The Formula: `Score = (0.4 * Semantic Vector) + (0.4 * BM25 Lexical) + (0.2 * Metadata)`

- **Why 0.4 for Semantic?** It provides the “intent” and context so we catch synonyms and related concepts.
 - **Why 0.4 for Lexical?** It acts as an anchor. If the AI hallucinates, BM25 forces it to bring back articles containing the exact words typed.
 - **Why 0.2 for Metadata?** We treat metadata as a “tie-breaker”. If we made it 0.8, the system would just return the absolute newest articles, even if they had nothing to do with the search.
 - **The “Zero Score Bug” Fix:** In early testing, BM25 would return a strict `0.0` if an exact word wasn’t found. Because our algorithm combined these, that `0.0` destroyed the total score. *Our Fix:* We implemented a “Floor Limit” of `0.001` for BM25. This ensures the math doesn’t break and allows FAISS (meaning) to pull the article to the top.
-

8. The Storage Architecture (The Database)

What we use: DuckDB

- **What is it?** An embedded SQL database specifically built for Analytics (OLAP).
 - **Why DuckDB?**
 - It is **Columnar**. Standard databases store data row-by-row (good for adding single users). Columnar databases store columns together (good for instantly aggregating thousands of metadata tags).
 - It is **Embedded**. It runs natively inside our Python backend.
 - **Why we rejected MySQL / PostgreSQL:** They require a heavy external server constantly running in the background. They are difficult to set up locally.
 - **Why we rejected MongoDB:** Document stores are great for unstructured data, but news metadata (time, source, category) is highly structured and benefits from strict SQL querying for analytics.
-

9. The Application Layer (Frontend & Backend)

Backend: Python via Flask/FastAPI

- **Why?** Python is the undisputed king of Data Science and AI. All our libraries (DuckDB, scikit-learn, FAISS, sentence-transformers) are native to Python.

Frontend: React (JavaScript/HTML/CSS)

- **Why?** React uses a virtual DOM to rebuild the UI instantly.
 - **The UI Problem we fixed:** Initially, when we booted the backend, loading the 120,000 vectors into RAM took a few seconds. Users thought the app was frozen/broken. We implemented React asynchronous loading states (spinners) to indicate backend readiness, drastically improving user experience.
-

10. How to explain the Results (Metrics)

We measure success using **Precision** and **Recall**. Here is exactly how to explain them verbally:

- **The Setup:** We manually wrote 50 test searches (e.g., "SpaceX launch", "European economy") and manually flagged which database articles *should* be the correct answers. This is our "Ground Truth".
- **Precision@10 (0.78):** When a user searches, the system gives them 10 top results. A score of 0.78 means that out of those 10 hits, almost 8 of them were highly relevant. *Concept:* "How much of what we returned is actually useful?"
- **Recall@20 (0.82):** Out of all the relevant articles in our massive 120k database, our system successfully found 82% of them within its top 20 guesses. *Concept:* "How much of the total truth did we successfully catch?"
- **F1-Score (0.80):** Simply the mathematical average of Precision and Recall. High F1 means the system is perfectly balanced—it doesn't spam bad results, and it rarely misses good results.